

SECURE PROGRAMMING

EXERCISE 3 SOLUTION REPORT

Configuring HTTPS, Secure Auditing, and Code Vulnerability Remediation

Submission Date:	June 02, 2026
------------------	---------------

Course:	Secure Programming - Assignment 3
---------	--------------------------------------

Status:	Task A Complete - In Progress
---------	----------------------------------

Report Outline

Section	Status	Page
A. Adding a Digital Certificate (HTTPS Setup)	COMPLETED	3
C. Analysis of <object> Tag Restrictions	PENDING	-
D. Codebase Vulnerability Remediation	PENDING	-
E, F, G. Advanced Cyber Security Concepts	PENDING	-

A. Adding a Digital Certificate (HTTPS Setup)

This section details the design, configuration, and implementation of a locally trusted HTTPS (SSL/TLS) environment for the **XSSApp** website. The implementation ensures that the server uses a logical domain name (**xssapp.local**) and that modern browsers (specifically Chrome 85 and above) recognize the SSL certificate as fully secure without displaying any warnings.

Why Standard Self-Signed Certificates Fail in Modern Chrome

Historically, browsers validated certificates using the **Common Name (CN)** field (e.g., CN=localhost). However, starting with Chrome 58 and strictly enforced in version 85+, Google deprecated and removed support for Common Name matching. Chrome now exclusively validates the server identity using the **Subject Alternative Name (SAN)** extension. If the SAN extension is missing or does not match the active hostname, Chrome raises a `NET::ERR_CERT_COMMON_NAME_INVALID` error, even if the certificate is signed by a trusted root authority.

Implementation Checklist & Walkthrough

1. Named Host Configuration

To satisfy the requirement of a 'logical/valid name', we mapped a custom domain to our local loopback interface. This was done by editing the Windows hosts file (`C:\Windows\System32\drivers\etc\hosts`) and adding the following entry mapping the local loopback to our logical hostname:

```
127.0.0.1 xssapp.local
```

2. Creating the OpenSSL Configuration (openssl.cnf)

To generate a certificate with the required SAN extension, we created an OpenSSL configuration file named `openssl.cnf`. This file explicitly requests the **subjectAltName** extension to contain both our logical domain name and the local loopback IP address (ensuring fallback compatibility):

```
[req]
distinguished_name = req_distinguished_name
x509_extensions = v3_req
prompt = no

[req_distinguished_name]
C = US
ST = State
L = City
O = Organization
CN = xssapp.local

[v3_req]
keyUsage = keyCertSign, cRLSign, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = xssapp.local
IP.1 = 127.0.0.1
```

3. Generating Key and Certificate Pair

Using the OpenSSL executable found inside the local Git installation, we ran the following command to generate a new 2048-bit RSA private key (`key.pem`) and a self-signed X.509 certificate (`cert.pem`) valid for 365 days using our configuration file:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout key.pem -out cert.pem -config openssl.cnf
```

4. Installing and Trusting the Certificate in Windows Store

Even with a valid SAN, Chrome will reject the certificate because it is self-signed and does not chain up to a trusted certificate authority. To establish trust, we imported the certificate (cert.pem) into the Windows **Trusted Root Certification Authorities** store for the active user using PowerShell:

```
Import-Certificate -FilePath .\cert.pem -CertStoreLocation Cert:\CurrentUser\Root
```

5. Modifying app.py to Serve HTTPS

Finally, we updated the Flask server's startup routine in app.py to dynamically scan for cert.pem and key.pem. If found, it automatically initializes the server in HTTPS mode using an SSL context, binding securely to port 5000:

```
def start_app():
    ...
    cert_path = "cert.pem"
    key_path = "key.pem"
    if not (os.path.exists(cert_path) and os.path.exists(key_path)):
        base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        cert_path = os.path.join(base_dir, "cert.pem")
        key_path = os.path.join(base_dir, "key.pem")
    if os.path.exists(cert_path) and os.path.exists(key_path):
        app.logger.info("Starting server with HTTPS...")
        app.run(host="0.0.0.0", port=5000, ssl_context=(cert_path, key_path))
    else:
        app.logger.info("Starting server with HTTP...")
        app.run(host="0.0.0.0", port=5000)
```